`torch.Tensor([1,0,0,0]).reshape(1,4)`. The result we get when multiplying two matrices depends on their dimensions and the order in which we multiply them. If we take an *A*: 1 × 4 matrix and multiply it by another matrix *B*: 4 × 1, then we get a 1 × 1 result, which is a *scalar* (a single number). This would be the *inner product* of two vectors (promoted to matrices), since the largest dimensions are sandwiched in between the two singlet dimensions. The outer product is just the reverse of this, where the two large dimensions are on the outside and the two singlet dimensions are on the inside, resulting in a $4 \times 1 \otimes 1 \times 4 = 4 \times 4$ matrix.

If we have two agents in Gridworld with individual actions [0,0,0,1] ("right") and [0,0,1,0] ("left"), their joint action can be computed by taking the outer product of these vectors. Here's how we do it in numpy:

```
>>> np.array([[0,0,0,1]]).T @ np.array([[0,1,0,0]])
array([[0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 1, 0, 0]])
```

The result is a 4 × 4 matrix, with a total of 16 elements as we would expect from our discussion in the previous section. The dimension of the result of the outer product between two matrices is dim(*A*) * dim(*B*), where *A* and *B* are vectors and "dim" refers to the size (dimension) of the vector. The outer product is the reason why the joint-action space grows exponentially. Generally, we need our neural network Q function to operate on inputs that are vectors, so since the outer product gives us a matrix result, we simply flatten it into a vector:

```
>>> z = np.array([[0,0,0,1]]).T @ np.array([[0,1,0,0]])
>>> z.flatten()
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0])
```

Hopefully, you can appreciate that the neighborhood Q-learning approach is not much more complicated than ordinary Q-learning. We just need to give it an additional input, which is the joint-action vector of each agent's nearest neighbors. Let's figure out the details by tackling a real problem.